WheelBot — Technical Specification & Build Plan

Status: Pre-build. Decisions locked in this doc; open questions called out explicitly. **Owner:** David **Companion to:** PolyTrader (LXC 104). Will run on the same Proxmox host but in a *separate* LXC.

---

1. Executive Summary

A long-running automated trading bot that executes the **options wheel strategy**: sell cash-secured puts (CSPs) on stocks we'd be willing to own; if assigned, sell covered calls (CCs) on those shares; collect premium continuously; recycle capital. Optimized for a small starting bankroll (⩽ $10k), Python-native, designed to mirror the PolyTrader architecture so deployment, ops, and mental model carry over.

Core decisions made up front:

Decision	Choice	Reason
Primary broker	**Tastytrade** (production)	Permanent refresh tokens (no headless-server pain), purpose-built for options sellers, mature Python SDK, sandbox available
Paper / dev broker	**Alpaca paper**	Free, instant, great for fast iteration on strategy logic; switch to Tastytrade sandbox before going live
Language / runtime	Python 3.11+, asyncio	Matches tastyware/tastytrade SDK, matches PolyTrader
Database	SQLite (v1) → Postgres (v2 if needed)	Single-file deploy, easy backup; rotate to Postgres only if multi-process
Deployment	Docker container inside dedicated LXC on Proxmox	Same pattern as PolyTrader (LXC 104). Use a NEW LXC, not 104.
Dashboard port	**8889**	Avoids collision with PolyTrader's 8888
Code base	**Fork wheel-it** for Alpaca prototype, **build native** for Tastytrade with broker abstraction	wheel-it gives ~40% of v1 for free; rewrite for production
LLM integration	Anthropic API (Opus 4.7 for daily screening, Haiku for high-frequency roll advisor calls)	Multi-model ensemble pattern as used in PolyTrader weather strategy

Hard non-goals for v1:

- No multi-leg spreads (verticals, condors, straddles). Wheel only.
- No 0DTE. 30–45 DTE band only.
- No fully autonomous "what should I trade" — universe is curated and gated.
- No directional bets. Income generation only.

---

2. Broker Selection — Detailed Comparison

Feature	Tastytrade	Tradier (Pro)	Schwab	Alpaca
Auth	OAuth2, **refresh token never expires**	Bearer token (simple)	OAuth2, **refresh token expires every 7 days**	API key/secret
Approval time	~1 day	~1 day	1–3 business days, sometimes rejected	Instant
Sandbox	Yes (`api.cert.tastyworks.com`, resets daily, 15-min delayed quotes)	Yes (`sandbox.tradier.com`, paper money + delayed)	Yes (synthetic accounts)	Yes (real-time, very high fidelity)
Multi-leg options	Yes	Yes (up to 4 legs)	Limited in most SDKs	Yes (newer)
Options commission	$1/leg open, $0/leg close, $10 cap	$0 with Pro plan ($10/mo)	$0.65/contract	$0
Python SDK quality	`tastyware/tastytrade` — async, typed, excellent	Multiple unofficial; REST is simple	`schwab-py`; multi-leg incomplete	Official `alpaca-py` — best DX
Rate limits	Reasonable; fails-on-too-many-failed-logins (8h IP block)	120/min standard, 600/min premium	~120/min	200/min free, 1000/min funded
Headless-server suitability	**Excellent** — refresh once, runs forever	Excellent — bearer token	**Poor** — must redo OAuth weekly	Excellent

Verdict for our use case: Tastytrade > Tradier > Alpaca > Schwab for live wheel automation. Schwab eliminated by the 7-day refresh token expiry — ops nightmare for a bot that needs to manage open positions across long weekends and vacations.

Implementation note: Build a `Broker` abstract base class so swapping is one file. Don't bake Tastytrade calls into the strategy layer.

---

3. Architecture & File Layout

Directory structure mirrors PolyTrader (`core/`, `data/`, `platforms/`, `strategies/`, `execution/`, `dashboard/`) so the mental model is the same:

```
/opt/wheelbot/                  # symlink to /mnt/wheelbot-storage/wheelbot
├── core/
│   ├── __init__.py
│   ├── broker.py               # Broker ABC: get_positions, get_quote,
place_order, get_chain, etc.
│   ├── lifecycle.py            # State machine for a wheel position
│   ├── models.py               # Pydantic: Position, Order, OptionContract,
WheelCycle
│   └── checkpoint.py           # Checkpoint logging helper (matches PolyTrader
pattern)
├── platforms/
│   ├── tastytrade_broker.py    # implements Broker for tastytrade SDK
│   ├── alpaca_broker.py        # implements Broker for alpaca-py
│   └── paper_broker.py         # in-memory mock for unit tests
├── data/
│   ├── chain.py                # Fetch & filter option chains, calc bid-ask
spread, OI
│   ├── greeks.py               # Black-Scholes + delta/theta/vega/IV calc
(fallback if broker doesn't supply)
│   ├── ivr.py                  # IV Rank / IV Percentile from rolling 52w
history
│   ├── earnings.py             # Earnings calendar (yfinance or Finnhub)
│   └── universe.py             # Load + validate universe.yaml
├── strategies/
│   ├── wheel.py                # Top-level orchestrator
│   ├── csp_selector.py         # Pick best CSP given config (delta band, DTE,
IVR, premium yield)
│   ├── cc_selector.py          # Pick best CC, with strike >= cost basis floor
│   ├── roll_advisor.py         # When tested, decide: roll for credit, take
assignment, or close
│   └── exit_manager.py         # Profit-take at 50%, time-exit at 7 DTE
├── execution/
│   ├── router.py               # place_order with retry, idempotency keys, dry-
run mode
│   ├── reconciler.py           # Periodic poll: detect fills, assignments,
expirations
│   └── kill_switch.py          # Honors manual stop file + daily loss cap
├── intelligence/               # LLM-powered components (optional, gated by
config)
│   ├── screener.py             # Daily candidate ranking (LLM)
│   ├── news_check.py           # Pre-trade news scan for catalysts
│   ├── roll_advisor_llm.py     # LLM second opinion on roll decisions
│   └── ensemble.py             # Multi-model voting (matches PolyTrader weather
```

```
pattern)
├── risk/
│   ├── sizing.py               # Per-position sizing, BP allocation, concurrent
caps
│   ├── exposure.py             # Sector / correlation tracking
│   ├── regime.py               # SPY 200-day, VIX level, choppiness —
pause/resume signals
│   └── limits.py               # Hard limits: max loss/day, max BP%, max DTE
├── dashboard/
│   ├── app.py                  # FastAPI server on :8889
│   ├── templates/              # Minimal HTML/HTMX, no SPA
│   └── static/
├── db/
│   ├── schema.sql              # Source of truth for SQLite schema
│   ├── migrations/             # Alembic-style or hand-rolled, numbered
│   └── repo.py                 # Repository pattern: positions_repo,
orders_repo, cycles_repo
├── config/
│   ├── config.yaml             # Defaults, committed to repo
│   ├── config.local.yaml       # Local overrides, gitignored, mirrors PolyTrader
pattern
│   ├── universe.yaml           # Tickers + per-ticker parameter overrides
│   └── secrets.env             # API keys, gitignored
├── tests/
│   ├── unit/
│   ├── integration/            # Hits broker sandbox
│   └── fixtures/               # Recorded option chain JSON for deterministic
tests
├── scripts/
│   ├── bootstrap_db.py
│   ├── ingest_history.py       # Build IVR rolling history
│   ├── manual_close.py         # Emergency close-everything CLI
│   └── replay_cycle.py         # Replay a past cycle from DB for debugging
├── docker-compose.yml
├── Dockerfile
├── pyproject.toml
└── README.md
```

Ops layout (matches PolyTrader exactly):

- LXC: new container, e.g. `LXC 105`, hostname `WheelBot`
- Code path: `/opt/wheelbot` → symlink to `/mnt/wheelbot-storage/wheelbot`
- Storage: dedicate ≥ 50 GB SSD for DB + chain history + logs

- Don't share storage volume with PolyTrader

---

4. The Wheel State Machine

Every position (one per ticker) lives in exactly one state. Transitions are atomic and logged.

IDLE
candidate selected
SCANNING
CSP order placed
CSP_PENDING
cancel/reject
filled
CSP_OPEN
50%-profit close
tested (delta>=.5)
ROLL_EVAL
roll
let-assign
CSP_CLOSED
ASSIGNED
→ IDLE
SHARES_HELD
CC_PENDING
filled

Error states (parallel to all of the above):

- `BROKER_DOWN` — API failures > N retries; pause new orders, keep monitoring
- `MANUAL_INTERVENTION` — anything ambiguous (partial fill, surprise corporate action, mismatch between local state and broker state); halt this position only
- `KILLED` — kill switch tripped; stop everything, hold existing positions

Implementation: state stored in `positions` table with `state_changed_at` timestamp and `state_change_reason` text column. Every transition writes a row to `state_log` table — full audit trail.

---

5. Wheel Parameters (Defaults)

Dial these in `config.yaml`. Every parameter has a per-ticker override slot in `universe.yaml`.

yaml

```yaml
wheel:
  # Strike selection
  csp_delta_min: 0.20
  csp_delta_max: 0.30
  cc_delta_min: 0.20
  cc_delta_max: 0.30
  cc_strike_floor: cost_basis     # never sell CC below cost basis

  # Time
  dte_min: 30
  dte_max: 45
  weekly_only: false              # if true, only trade contracts with weekly expiri

  # Entry gates
  ivr_min: 30                     # don't sell premium when premium is cheap
  bid_ask_spread_max_pct: 5       # skip illiquid contracts (spread > 5% of mid)
  open_interest_min: 500
  volume_min: 100                 # contract daily volume

  # Earnings handling
  earnings_blackout_days_before: 5
  earnings_blackout_days_after: 2
  earnings_low_delta_override: 0.10  # if you must trade across earnings, use very l

  # Exit / management
  profit_close_pct: 50            # close at 50% max profit
  time_close_dte: 7               # close any position with <= 7 DTE remaining
  roll_trigger_delta: 0.50        # consider roll when short option's delta hits thi
  roll_only_for_credit: true      # never debit-roll

  # Position sizing
  max_position_pct_of_account: 30 # no single ticker > 30% of buying power
  max_concurrent_positions: 4     # tunable per account size
  buying_power_floor_pct: 20      # always keep 20% BP free

risk:
  daily_loss_kill_switch_pct: 5   # if account drops 5% in a day, halt new trades
  consecutive_losses_pause: 3     # 3 losses in a row → pause for human review

regime:
  enabled: true
  spy_above_sma_days: 200         # require SPY > 200-day SMA to enter new CSPs
  vix_max: 35                     # halt new CSPs above this
```

```yaml
  pause_on_vix_spike_pct: 30      # if VIX +30% in a day, pause

intelligence:
  llm_screener_enabled: true
  llm_news_check_enabled: true
  llm_roll_advisor_enabled: false  # turn on after collecting 3+ months of data
  llm_provider: anthropic
  screener_model: claude-opus-4-7
  news_check_model: claude-haiku-4-5
  roll_advisor_models:             # ensemble vote
    - claude-opus-4-7
    - claude-haiku-4-5
  ensemble_quorum: 2               # need N agreeing models to act
```

`config.local.yaml` overrides per-section, same as PolyTrader. Example local override during the live-test phase:

yaml

```yaml
wheel:
  max_concurrent_positions: 1     # start with one position
  max_position_pct_of_account: 100  # full deployment in single position during test
risk:
  daily_loss_kill_switch_pct: 3   # tighter stop while learning
intelligence:
  llm_screener_enabled: false     # turn this on after manual screening proves relia
```

---

6. Universe Definition

`config/universe.yaml` — curated. **Never** auto-add tickers without human review.

yaml

```yaml
```

```yaml
# Tier 1 — Safe core for small accounts (under $30/share, large-cap, dividend-paying
- symbol: F
  name: Ford Motor Company
  tier: 1
  notes: "Liquid, dividend, low IV but reliable. Good for first live trades."
  overrides:
    csp_delta_max: 0.25         # tighter delta given low premium
- symbol: SOFI
  name: SoFi Technologies
  tier: 1
  notes: "Higher IV fintech. Good monthly premium."
- symbol: BAC
  name: Bank of America
  tier: 1
  notes: "Low IV but tight spreads, very liquid."
- symbol: NOK
  name: Nokia
  tier: 1

# Tier 2 — Higher beta, higher premium, requires bigger BP cushion
- symbol: RIVN
  name: Rivian
  tier: 2
  overrides:
    max_position_pct_of_account: 20   # tighter position limit
- symbol: PLTR
  name: Palantir
  tier: 2
- symbol: PLUG
  name: Plug Power
  tier: 2

# Tier 3 — Trade only when LLM screener AND human approve
- symbol: HOOD
  name: Robinhood Markets
  tier: 3
  notes: "Wheel the broker. Approved one cycle at a time."

# Banned list — explicitly never trade these even if screener suggests
banned:
  - GME    # too erratic
```

```
  - AMC    # too erratic
  - any biotech under $20  # binary FDA risk
```

`tier` is enforced in code: `tier 1` can be auto-traded, `tier 2` requires LLM screener + risk-checks pass, `tier 3` requires manual human approval per cycle (post a Slack/dashboard prompt).

---

7. Database Schema (SQLite)

sql

```sql
-- Current state per (account, symbol)
CREATE TABLE positions (
    id INTEGER PRIMARY KEY,
    account_id TEXT NOT NULL,
    symbol TEXT NOT NULL,
    state TEXT NOT NULL,                -- IDLE, CSP_OPEN, SHARES_HELD, CC_OPEN, etc
    shares INTEGER NOT NULL DEFAULT 0,
    cost_basis REAL,                    -- avg cost per share, premium-adjusted
    current_cycle_id INTEGER,           -- FK to wheel_cycles
    state_changed_at DATETIME NOT NULL,
    state_change_reason TEXT,
    UNIQUE(account_id, symbol)
);

-- Every order ever submitted (CSP, CC, BTC, BTO)
CREATE TABLE orders (
    id INTEGER PRIMARY KEY,
    account_id TEXT NOT NULL,
    symbol TEXT NOT NULL,
    cycle_id INTEGER,
    broker_order_id TEXT UNIQUE,        -- broker's ID (idempotency)
    client_order_id TEXT UNIQUE,        -- our UUID (dedupe)
    order_type TEXT NOT NULL,           -- SELL_TO_OPEN, BUY_TO_CLOSE, etc.
    contract_symbol TEXT,               -- OCC option symbol or stock ticker
    strike REAL,
    expiration DATE,
    option_type TEXT,                   -- PUT, CALL, NULL for stock
    quantity INTEGER NOT NULL,
    limit_price REAL,
    fill_price REAL,
    status TEXT NOT NULL,               -- PENDING, FILLED, PARTIAL, CANCELLED, REJE
    placed_at DATETIME NOT NULL,
    filled_at DATETIME,
    raw_request JSON,                   -- full broker request body
    raw_response JSON                   -- full broker response
);

-- A wheel cycle = CSP open → ... → final close. One symbol can have many cycles ove
CREATE TABLE wheel_cycles (
    id INTEGER PRIMARY KEY,
    account_id TEXT NOT NULL,
    symbol TEXT NOT NULL,
    started_at DATETIME NOT NULL,
```

```
    ended_at DATETIME,
    initial_csp_strike REAL,
    initial_csp_premium REAL,
    initial_capital_at_risk REAL,
    final_pnl REAL,                    -- realized at cycle end
    final_pnl_pct REAL,
    cycle_outcome TEXT,                -- CSP_EXPIRED, CC_CALLED_AWAY, MANUAL_CLOSE
    days_held INTEGER,
    n_orders INTEGER
);

-- State transition log (full audit)
CREATE TABLE state_log (
    id INTEGER PRIMARY KEY,
    position_id INTEGER NOT NULL,
    from_state TEXT,
    to_state TEXT NOT NULL,
    reason TEXT,
    triggered_by TEXT,                 -- ORDER_FILL, RECONCILER, MANUAL, KILL_SWIT
    metadata JSON,
    created_at DATETIME NOT NULL
);

-- Daily LLM screener output, even if not acted upon
CREATE TABLE candidates (
    id INTEGER PRIMARY KEY,
    run_date DATE NOT NULL,
    symbol TEXT NOT NULL,
    score REAL,
    rank INTEGER,
    rationale TEXT,
    ivr REAL,
    iv_pct REAL,
    price REAL,
    bp_required REAL,
    suggested_strike REAL,
    suggested_dte INTEGER,
    raw_llm_response JSON,
    acted_on BOOLEAN DEFAULT FALSE
);

-- Daily regime snapshot
CREATE TABLE regime_snapshots (
    id INTEGER PRIMARY KEY,
```

```
    snapshot_date DATE UNIQUE NOT NULL,
    spy_close REAL,
    spy_sma_200 REAL,
    spy_above_sma BOOLEAN,
    vix_close REAL,
    vix_change_pct REAL,
    choppiness REAL,
    regime TEXT,                        -- BULL_TREND, NEUTRAL, BEAR_TREND, HIGH_VOL
    csps_allowed BOOLEAN,
    notes TEXT
);

-- Audit trail of LLM decisions (so we can backtest "would the LLM have helped?")
CREATE TABLE llm_decisions (
    id INTEGER PRIMARY KEY,
    decision_type TEXT NOT NULL,        -- SCREEN, NEWS_CHECK, ROLL_ADVISE
    context JSON,
    model TEXT,
    response JSON,
    decision TEXT,
    confidence REAL,
    acted_on BOOLEAN,
    outcome TEXT,                       -- backfilled later when result known
    created_at DATETIME NOT NULL
);

-- IV history for IV Rank/Percentile calc (rolling 52w per symbol)
CREATE TABLE iv_history (
    id INTEGER PRIMARY KEY,
    symbol TEXT NOT NULL,
    snapshot_date DATE NOT NULL,
    iv_30d REAL,                        -- ATM 30-day IV
    UNIQUE(symbol, snapshot_date)
);
```

Why SQLite for v1: simple file, easy nightly backup to the storage SSD, no separate process. If multiple containers ever need to read this DB simultaneously, migrate to Postgres — the repository pattern in `db/repo.py` makes that swap mechanical.

---

8. Risk Management — Hard Rules

These are encoded as preconditions in `risk/limits.py` and checked **before every order**

submission. Any failure raises `RiskCheckFailed` and the order is not placed (and the failure is logged loudly).

1. **Buying power floor:** account must have ≥ `buying_power_floor_pct` (default 20%) free *after* the proposed order.
2. **Per-position cap:** notional value of any single ticker's position (cash secured for puts, share value for stock) ≤ `max_position_pct_of_account`.
3. **Concurrent positions cap:** count of non-IDLE positions ≤ `max_concurrent_positions`.
4. **Earnings blackout:** ticker has no earnings event within `earnings_blackout_days_before` of expiration date.
5. **Strike floor (CC only):** strike ≥ cost basis. No exceptions in v1.
6. **Liquidity gates:** OI ≥ `open_interest_min`, daily volume ≥ `volume_min`, bid-ask spread ≤ `bid_ask_spread_max_pct` of mid.
7. **Regime gate:** if `regime.enabled` and current snapshot has `csps_allowed = false`, no new CSPs (existing positions still managed).
8. **Daily kill switch:** if account dropped > `daily_loss_kill_switch_pct` since session open, halt all new orders for the day.
9. **Consecutive losses pause:** N losing cycles in a row → halt new orders and post a "needs human review" message to dashboard.
10. **Manual stop file:** existence of `/opt/wheelbot/STOP` halts all new orders. Reconciler still runs. (This is the panic button. Touching this file is the safest emergency action.)

Important: rules 1–7 prevent bad trades. Rules 8–10 stop a runaway. Both are necessary — they fail in different ways.

---

9. LLM Integration Architecture

The LLM is **not** in the trade execution hot path. It's an advisor that produces structured outputs the rule-based engine can use, ignore, or reject. Same philosophy as your PolyTrader weather strategy ensemble.

9.1 Daily Screener (`intelligence/screener.py`)

Runs once daily, pre-market.

Input: Universe.yaml minus banned, filtered to `tier=1,2`. For each, fetch: current price, IVR, IV percentile, recent price action (5/20/50 SMAs), upcoming earnings date, recent news headlines (last 7 days).

Prompt structure: Sends Claude Opus a structured payload. Asks for a ranked shortlist with rationale per ticker. Returns JSON.

Output: Top N candidates written to `candidates` table. The strategy layer pulls from this table when deciding what to trade.

Hard rule: screener output is a *suggestion*. Risk gates and per-ticker tier rules still apply. Tier 2 candidates that pass the screener may be auto-traded; tier 3 always require human ack.

9.2 Pre-Trade News Check (`intelligence/news_check.py`)

Before placing any new CSP, fetch news from the last 48h on the ticker, send to Claude Haiku (cheaper, fast), ask: "Is there a catalyst in this news that would make selling a 30-day put unwise?" Returns `proceed | caution | block` with rationale.

If `block` → cancel the order, log decision, surface in dashboard. If `caution` → reduce size by 50%, proceed. If `proceed` → normal order.

9.3 Roll Advisor (`intelligence/roll_advisor_llm.py`) — Phase 2

When a short option goes ITM (delta ⩾ roll_trigger_delta), the rule-based `strategies/roll_advisor.py` produces its recommendation (roll for credit / let assign / close). The LLM ensemble then independently evaluates the same situation. **If they disagree, halt and post for human review.**

This is intentionally conservative. The point isn't to let the LLM override deterministic rules — it's to use LLM as a "second pair of eyes" that catches situations the rule-based logic didn't anticipate.

9.4 Ensemble Pattern (matches PolyTrader weather strategy)

python

```python
async def llm_ensemble_vote(prompt, models=["claude-opus-4-7", "claude-haiku-4-5"]):
    responses = await asyncio.gather(*[query(m, prompt) for m in models])
    decisions = [parse_decision(r) for r in responses]
    if all_agree(decisions):
        return decisions[0], "unanimous"
    if quorum_reached(decisions, n=config.ensemble_quorum):
        return majority_decision(decisions), "majority"
    return None, "no_consensus"  # halt
```

9.5 Audit Trail

Every LLM call writes to `llm_decisions` table with full context, response, and whether we acted on it. After 3 months you can run `SELECT outcome FROM llm_decisions WHERE acted_on =`

`TRUE` and actually measure if the LLM helped. Without this you're guessing.

---

10. Reconciliation Loop

Bot can crash, broker can lag, fills can happen overnight (assignments). The reconciler is the source of truth.

Runs every 5 minutes during market hours, every 30 minutes off-hours.

For each `account_id`:

1. Fetch current positions from broker.
2. Fetch all orders updated since last reconcile.
3. Compare to local DB:
   - New fill → update order, transition state, log
   - Assignment → transition `CSP_OPEN → SHARES_HELD`, update cost basis to `strike - premium_collected`
   - Expiration (worthless) → transition `CSP_OPEN → IDLE` (or `CC_OPEN → SHARES_HELD`), close cycle if applicable
   - Called away → transition `CC_OPEN → IDLE`, close cycle, calc realized P&L
   - Mismatch (broker shows position we don't have, or vice versa) → flag `MANUAL_INTERVENTION`, do NOT auto-correct
4. Recompute `cost_basis` after every state change.

The reconciler is the only thing that writes state changes for fills. The order router only writes `PENDING`. This separation prevents double-counting and means bot crashes mid-order are recoverable.

---

11. Dashboard (Phase 1 minimal)

FastAPI on `:8889`, server-side HTMX. No SPA, no React. Five views:

- / — current positions table: symbol, state, days in state, current P&L, days to expiration
- /**cycles** — all wheel cycles (closed and open), filterable, sorted by P&L
- /**candidates** — most recent screener output
- /**orders** — recent orders, with broker request/response inspectable

- **/risk** — current BP usage, regime indicators, kill switch status, button to toggle manual stop file

Authentication: HTTP Basic with creds from `secrets.env`. Not internet-exposed; bind to `127.0.0.1` and reach via Tailscale or SSH tunnel from your network.

---

12. Phased Rollout Plan

Phase	Duration	What	Capital	Goal
0. Spec & scaffold	1 wk	This doc → repo skeleton, DB schema, broker ABC, Alpaca paper adapter	$0	Repo runnable, paper auth working
1. Logic on Alpaca paper	2–3 wks	Implement wheel state machine + selectors + reconciler. Trade single ticker (F).	$0	One full cycle (CSP → assign → CC → called away) executed end-to-end on paper
2. Tastytrade sandbox	2 wks	Implement Tastytrade adapter. Re-run logic against sandbox.	$0	Same cycles execute identically against Tastytrade sandbox. Sandbox quotes are 15-min delayed — verify reconciler handles this.
3. Live single-ticker	4 wks	Live $1–2k on Tastytrade, single position, F or SOFI. Manual approval before each trade.	$1–2k	Survive a full month with no operational issues. Get felt experience with assignment, rolling, red days.
4. Live small portfolio	4 wks	2–3 concurrent positions, tier 1 only. Auto-approval enabled.	$3–5k	Validate concurrent management, BP allocation logic, dashboard usability
5. LLM screener live	4 wks	Turn on daily screener, expand to tier 2. Compare LLM-recommended vs rule-only outcomes.	$5–8k	Measure: did the LLM help?

Phase	Duration	What	Capital	Goal
6. Regime + news check	ongoing	Layer regime filter and pre-trade news check. Iterate on parameters with real P&L data.	scale gradually	Build a year of live data before trusting the bot with serious capital

Scale gates between phases:

- Phase 3 → 4: zero unhandled errors, zero "what just happened" moments, > 80% of fills reconciled within 5 min
- Phase 4 → 5: positive total P&L over a 4-week period (bear in mind 4 weeks is very short — this is an operational gate, not a strategy validation)
- Phase 5 → 6: LLM screener picks materially outperform random selection from same universe (track this in `llm_decisions` table)

Hard rule: do not skip phases. The mistake is not the algorithm. The mistake is operational issues during real money trading. Every phase exposes a different class of bug.

---

13. Build Order — Tickets for Claude Code

Roughly the order I'd hand things to Claude Code. Each is sized to ~1–4 hours of focused work.

Sprint 1 — Foundation

1. Bootstrap repo: directory structure, `pyproject.toml`, `Dockerfile`, `docker-compose.yml`, README skeleton.
2. Implement `db/schema.sql` exactly as specified. `scripts/bootstrap_db.py` to create the SQLite file.
3. Implement Pydantic models in `core/models.py` matching DB schema.
4. Repository pattern in `db/repo.py` — one class per table, async, fully typed.
5. Config loader: read `config.yaml`, overlay `config.local.yaml` per-section, load `secrets.env`.
6. Checkpoint logger in `core/checkpoint.py` matching PolyTrader style — every meaningful step logs `[CHECKPOINT] step=X status=ok/fail context={...}`.

Sprint 2 — Broker Abstraction

7. `core/broker.py` ABC: `get_account()`, `get_positions()`, `get_quote()`,

`get_option_chain()`, `place_order()`, `cancel_order()`, `get_orders_since()`. All async.

8. `platforms/paper_broker.py`: in-memory mock for unit tests.
9. `platforms/alpaca_broker.py`: implement against Alpaca paper. Use `alpaca-py`.
10. Integration test: place a paper CSP order through the abstraction, verify it appears in broker.

Sprint 3 — Strategy Core

11. `data/chain.py`: fetch + filter chain by DTE, delta, OI, spread.
12. `data/greeks.py`: BS pricing + Greeks (fallback when broker doesn't supply).
13. `data/ivr.py`: IVR/IV percentile from `iv_history`. Backfill script: `scripts/ingest_history.py`.
14. `strategies/csp_selector.py`: given symbol + config, return best CSP contract or `None`.
15. `strategies/cc_selector.py`: given symbol + cost basis + config, return best CC contract or `None`.
16. `strategies/wheel.py`: top-level orchestrator. Reads positions, dispatches to selectors per state.

Sprint 4 — Execution & Reconciliation

17. `execution/router.py`: `place_order` with idempotency (client_order_id), retry with exponential backoff, dry-run mode.
18. `execution/reconciler.py`: 5-min loop, fetch broker state, diff vs local, transition states. **This is the most critical module.** Test this thoroughly.
19. `execution/kill_switch.py`: check stop file, daily P&L, consecutive losses. Returns bool.
20. `risk/limits.py`: every gate from §8. Test each in isolation.

Sprint 5 — Operations

21. `dashboard/app.py`: FastAPI + HTMX views from §11.
22. `scripts/manual_close.py`: CLI to flatten one ticker or everything.
23. `scripts/replay_cycle.py`: replay a closed cycle from DB (useful for debugging).
24. Logging: structured JSON to file + console. Logrotate config (you noted this was a TODO on PolyTrader; do it from day 1 here).
25. Backup script: nightly cron, gzip the SQLite file to `/mnt/wheelbot-storage/backups/YYYY-MM-DD.sql.gz`, keep 30 days.

Sprint 6 — Tastytrade & Live Prep

26. `platforms/tastytrade_broker.py` against `tastyware/tastytrade` SDK.
27. OAuth bootstrap script: one-time, get refresh token, write to `secrets.env`.
28. Run integration test suite against Tastytrade sandbox.
29. Reconciler test: induce broker-vs-local mismatch, confirm `MANUAL_INTERVENTION` flag.

Sprint 7 — Intelligence Layer

30. `intelligence/ensemble.py`: multi-model voting helper.
31. `intelligence/screener.py`: daily screener prompt + JSON parsing + write to `candidates`.
32. `intelligence/news_check.py`: pre-trade news fetch + LLM check.
33. `risk/regime.py`: SPY 200d, VIX, choppiness — daily snapshot to `regime_snapshots`.

Sprint 8 — Polish

34. Phase-2 LLM roll advisor (only after we have data).
35. Per-ticker per-cycle backtester: replay a cycle's market conditions through the strategy code with current params, see if outcome would have been different.
36. Slack/Discord webhook for state-change alerts and risk events.

---

14. Open Questions To Resolve Before Build

1. **Account at Tastytrade:** options approval level? Need at least Level 2 (sells covered + cash-secured). Apply now — it can take a few days.
2. **PDT rule:** if your live account drops below $25k of equity, you're capped at 3 day-trades per 5 days. Wheel is *not* day-trading by design (we hold ⩾ 1 day), but a same-day open-and-close to take 50% profit on a CSP could count. Decide: never close same day, or require 4th-party confirmation when at the limit.
3. **IRA vs taxable:** taxable account allows margin (helps with BP). IRA is cash-only but tax-advantaged. Wheel income is short-term capital gains in taxable. Worth talking to a tax person before scaling.
4. **Earnings calendar source:** Finnhub free tier is generous; yfinance is brittle. Pick one and stick with it.

5. **Where do new tickers come from?** v1 has a hand-curated universe. Long term: do we want LLM to *propose* additions to universe.yaml that you then approve?

---

15. Things I'm Deliberately Not Doing in v1

Documenting these so they don't creep in mid-build.

- **No multi-leg spreads.** Iron condors, verticals, etc. are tempting but multiply risk and complexity. Wheel only.
- **No strike rolling between expirations.** v1 rolls only same expiration. Multi-expiration roll math is a future feature.
- **No early assignment handling beyond reconciler detection.** Early assignment of an ITM put is rare but real. v1 just detects and transitions to SHARES_HELD; v2 might attempt to immediately sell a CC same day.
- **No portfolio Greeks.** Tracking aggregate delta, vega, theta across positions is useful but not in v1.
- **No tax-lot optimization.** If we're called away on shares, broker handles lot selection.
- **No automatic position averaging.** If a stock drops, the bot does NOT automatically sell more puts at lower strikes ("doubling down"). Position management is one cycle at a time.
- **No leverage / portfolio margin.** Cash secured only.

---

16. Cross-cutting reminders

- **Checkpoint everything.** Same convention as PolyTrader: every step logs a `[CHECKPOINT] step_name status` line. When something goes wrong at 2am, the log is the first thing you read.
- **Fail loudly.** Schema mismatch, unexpected field shape, missing data — raise, don't paper over. (Same lesson as the FCC ASR null-island bug.)
- **Geodatabase-style I/O for state:** SQLite is the source of truth. Don't compute state from in-memory vars that survive only as long as the process. Reconciler reads from broker → writes to DB → strategy reads from DB.
- **Don't share container with PolyTrader.** Friday options expirations cluster CPU; PolyTrader's 5-min crypto loop should not contend.
- **Ctrl+W will close your Proxmox web console** — same lesson as PolyTrader. SSH from a real terminal for serious work.

Appendix A — Useful References

- **Python SDKs:** tastyware/tastytrade, alpaca-py
- **Reference implementations to study:** brndnmtthws/thetagang (IBKR-based, mature), vahagn-madatyan/wheel-it (Alpaca-based, simpler), alpacahq/options-wheel (official Alpaca template)
- **API docs:** Tastytrade developer portal, Tradier API docs, Alpaca options docs
- **Strategy reference:** Alpaca's wheel strategy walkthrough

---

End of spec. v0.1 — David, May 2026.